

1. ¿Qué es un flujo (Stream)?

Un **flujo** es un mecanismo que usa Java para **leer o escribir datos**.

Piensa en él como una **tubería**:

- Por un lado entra la información
- Por el otro lado sale
- Tu programa está en medio

Sirve para comunicarse con:

- Archivos
 - Teclado
 - Red
 - Memoria
-

2. Algoritmo universal de los flujos

Siempre que trabajes con streams, seguirás este patrón:

1. Abrir el flujo
2. Procesar (leer o escribir)
3. Cerrar el flujo

Esto es MUY importante. Si no cierras el flujo → puedes tener errores o pérdida de datos.

3. Flujos de bytes

Los flujos de bytes trabajan con datos **binarios (0 y 1)**.

Clases principales:

- `InputStream` → leer
- `OutputStream` → escribir

Son clases abstractas, no se usan directamente.

4. Leer un fichero con FileInputStream

Ejemplo: leer un archivo byte a byte

```
import java.io.FileInputStream;
import java.io.IOException;

public class LeerArchivo {
    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("archivo.txt");

            int dato;

            while ((dato = fis.read()) != -1) {
                System.out.print((char) dato);
            }

            fis.close();

        } catch (IOException e) {
            System.out.println("Error al leer el archivo");
        }
    }
}
```

Explicación:

- `fis.read()` → lee un byte
 - Devuelve un número (0–255) o `-1` si termina
 - Convertimos a `char` para mostrarlo
-

5. Escribir en un fichero con FileOutputStream

Ejemplo: escribir texto en un archivo

```
import java.io.FileOutputStream;
import java.io.IOException;

public class EscribirArchivo {
    public static void main(String[] args) {
        try {
            FileOutputStream fos = new FileOutputStream("archivo.txt");

            String texto = "Hola alumnos";
```

```

        fos.write(texto.getBytes());

        fos.close();

    } catch (IOException e) {
        System.out.println("Error al escribir el archivo");
    }
}
}

```

Explicación:

- `getBytes()` → convierte texto a bytes
- `write()` → escribe en el archivo

6. Sobrescribir vs añadir contenido

Por defecto, `FileOutputStream` **sobrescribe** el archivo.

Para **añadir contenido**:

```
FileOutputStream fos = new FileOutputStream("archivo.txt", true);
```

7. Ejemplo completo: copiar un archivo

Muy importante para entender streams

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopiarArchivo {
    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("origen.txt");
            FileOutputStream fos = new FileOutputStream("destino.txt");

            int dato;

            while ((dato = fis.read()) != -1) {
                fos.write(dato);
            }
        }
    }
}

```

```
        fis.close();
        fos.close();

        System.out.println("Archivo copiado correctamente");

    } catch (IOException e) {
        System.out.println("Error en la copia");
    }
}
}
```

8. Buenas prácticas (MUY importante para alumnos)

Usar try-with-resources (forma moderna)

```
import java.io.FileInputStream;
import java.io.IOException;

public class Ejemplo {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("archivo.txt")) {

            int dato;
            while ((dato = fis.read()) != -1) {
                System.out.print((char) dato);
            }

        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

- ✓ No hace falta `close()`
 - ✓ Java lo cierra automáticamente
-

9. Problema típico (explicación didáctica)

- ✗ Leer byte a byte es lento
 - ✓ Solución → usar buffers (lo verán en el Bloque 2)
-

Resumen

- Un **Stream** conecta tu programa con datos externos
- Siempre sigue:
abrir → procesar → cerrar
- **InputStream** → leer
- **OutputStream** → escribir
- **FileInputStream** y **FileOutputStream** → trabajar con archivos
- Se trabaja **byte a byte**

EJERCICIOS

1. Leer archivo byte a byte

Lee `texto.txt` y muéstralo por pantalla usando `FileInputStream`.

2. Copiar archivo

Copia `origen.txt` a `copia.txt` usando bytes.

3. Contar bytes

Lee un archivo y muestra cuántos bytes tiene.

4. Buscar carácter

Cuenta cuántas veces aparece la letra `a` en un archivo.

5. Reemplazo básico

Copia un archivo reemplazando todos los espacios por `-`.

6. Comparar dos archivos

Comprueba si dos archivos tienen exactamente los mismos bytes.

7. Invertir contenido

Lee un archivo y escríbelo al revés (byte a byte).

8. Filtro de caracteres

Copia un archivo eliminando los caracteres de control.

9. Lectura parcial

Lee solo los primeros 50 bytes de un archivo.

10. Concatenar archivos

Une dos archivos en uno solo.